

Reviewer Pack — Minimal Local Indefinite Integral over Elliptic Curves and C...

Entry 5ec0d789738d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 07:33:16 UTC

Minimal Local Indefinite Integral over Elliptic Curves and Communication Complexity Rank

Entry ID: 5ec0d789738d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 07:33:16 UTC

1. Conjecture

Field A (mathematical branch): Elliptic Curve Arithmetic **Field B** (complexity object): Communication Complexity

Statement:

For any given boolean function f , the minimal local indefinite integral (LII) of its characteristic polynomial over an elliptic curve with complex multiplication is linearly correlated with the communication complexity rank $r(f)$, such that $\text{LII}(f) = \Theta(r(f))$.

Rationale (proposer's reasoning):

Elliptic curves provide a rich algebraic structure for studying number-theoretic properties, and their integration may encode computational complexities in a novel way. The local indefinite integral captures the arithmetic behavior of functions on elliptic curves, which could potentially reflect the complexity of communication tasks.

Taxonomy category: elliptic_curve_arithmetic_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2e2e86832ef5ad5c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture that LII of characteristic polynomial over elliptic curves is linearly correlated with communication complexity rank, support is indicated if the Pearson correlation coefficient across 30 seeds is ≥ 0.8 and the mean absolute difference between $\text{LII}(f)$ and $r(f)$ for all functions f is ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local indefinite integral" AND "elliptic curve arithmetic"
- "communication complexity rank" AND characteristic polynomial" AND "elliptic curve" -
"integral over elliptic curves" AND communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=241.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def characteristic_polynomial(f):
        n = len(f)
        poly = [0] * (n + 1)
        poly[0] = 1
        for i in range(n):
            if f[i]:
                poly = add_polynomials(poly, multiply_polynomial(poly, [1, -2*i-1]))
        return poly

    def add_polynomials(p1, p2):
        result = [0] * max(len(p1), len(p2))
        for i in range(len(p1)):
            result[i] += p1[i]
        for i in range(len(p2)):
            result[i] += p2[i]
        return result

    def multiply_polynomial(p1, p2):
        result = [0] * (len(p1) + len(p2) - 1)
        for i in range(len(p1)):
            for j in range(len(p2)):
                result[i + j] += p1[i] * p2[j]
        return result
```

```

def local_indefinite_integral(poly):
    n = len(poly)
    integral = [0] * (n + 1)
    integral[0] = poly[0]
    for i in range(1, n):
        integral[i] = poly[i] / i
    integral[n] = poly[n]
    return integral

def communication_complexity_rank(f):
    n = len(f)
    rank = 0
    for i in range(n):
        if f[i]:
            rank += 1
    return rank

metric_name = "LII"
instances_tested = 30
n_max = 40
conjecture_holds = True
counterexample = ""

total_lii = 0
total_diff = 0

for _ in range(instances_tested):
    n = random.randint(5, n_max)
    f = generate_boolean_function(n)
    poly = characteristic_polynomial(f)
    lii = local_indefinite_integral(poly)
    rank = communication_complexity_rank(f)

    total_lii += abs(lii[0])
    total_diff += abs(lii[0] - rank)

mean_lii = total_lii / instances_tested
mean_diff = total_diff / instances_tested

if mean_diff > 3:
    conjecture_holds = False
    counterexample = "mean_diff_exceeds_3"

return {
    "metric_name": metric_name,
    "metric_value": mean_lii,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []

```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_lii = sum(r["metric_value"] for r in results) / len(results)
std_dev = math.sqrt(sum((r["metric_value"] - mean_lii) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_lii} std={std_dev} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_lii} std={std_dev} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"mean_diff_exceeds_3\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to provide a Pearson correlation coefficient and mean absolute difference as required by the support condition. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20288
2	propose	ollama_remote	glm4:latest	0	0	23846
3	propose	ollama_remote	glm4:latest	0	0	21372
4	propose	ollama_remote	glm4:latest	0	0	12370
5	preregistration	ollama_remote	glm4:latest	0	0	9435
6	novelty	ollama_remote	glm4:latest	0	0	8240
7	novelty	ollama_remote	glm4:latest	0	0	8737
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19635

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18833
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13536
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	50766
12	judge	ollama_remote	glm4:latest	0	0	31153

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 238211 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in research/audit/5ec0d789738d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/5ec0d789738d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5ec0d789738d.tar.gz (if generated)